

ATLAS Simulator Physics

How the frames, attitude representation, state, physics engines and jetfoil aerodynamics are put together in this simulator: what each one means physically, the equations the code actually uses, where it lives, and what is still an assumption.

World frame

in use

Flat, non-rotating NED treated as inertial. Earth rotation is 4 orders of magnitude below what matters here.

Quaternions

in use

Body-to-NED unit quaternion, scalar first, propagated at 1 kHz and renormalised every step.

State vector

in use

13 rigid-body states plus one spool state per fan (22 for V3). Contacts are algebraic, not states.

JSBSim

cross-check backend

Optional second integrator for frames, contact and integration. Same rotor and aero code. Not installed on this Mac.

Project Chrono

not present

No code or notes reference it. Its natural place is contact and flexible-structure dynamics.

Coanda and enclosure

simplified

Jet turned to the foil exit angle with a linear loss and ram drag. No attachment, separation or ground effect.

PLAIN-LANGUAGE EXPLAINERS IN THIS REPORT

Body-to-NED unit quaternion

Hover frame and rotor frame

Reading the equations of motion

Parallel-axis theorem

Reading the sensor equations

Hamilton product

Pitch rotation between frames

Fan spool and its lag

Leg contact is algebraic

FGFDMExec

The 90° singularity

The flow gyro frame mismatch

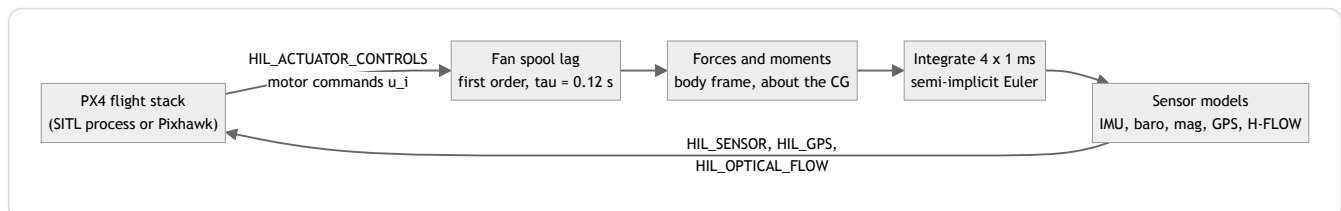
"Translation is written in NED"

Reading the integrator

0

One simulation step

The simulator is a loop that stands in for the physical world around a real PX4 flight stack. PX4 runs either as software on the Mac (SITL) or on the Pixhawk itself (HITL). Each loop iteration lasts one sensor period of 4 ms (250 Hz) and contains four physics substeps of 1 ms.



In SITL the two sides run in **lockstep**: every sensor packet advances PX4's clock by 4 ms, and the simulator waits for the matching motor command before stepping again, so a slow computer slows the whole world down instead of making it inaccurate. In HITL there is no lockstep; the board runs on its own clock and the simulator paces itself to real time.

Loop: `airframe_designer/sim/simulator.py` (`step_once`). Rigid body:
`airframe_designer/dynamics/rigid_body.py` .

1

World and inertial frame

Why an inertial frame is needed at all

Newton's second law, $\mathbf{F} = m\mathbf{a}$, and Euler's rotational equation only hold when acceleration is measured relative to an *inertial* frame: one that is not itself accelerating or rotating. Any rotating frame adds apparent forces. The connection between a vector's rate of change seen from two frames is the transport theorem: for any vector $\mathbf{b}$ and a frame $\mathbf{B}$ rotating at $\boldsymbol{\omega}_{B/I}$ relative to the inertial frame $\mathbf{I}$,

$$\left. \frac{d\mathbf{b}}{dt} \right|_I = \left. \frac{d\mathbf{b}}{dt} \right|_B + \boldsymbol{\omega}_{B/I} \times \mathbf{b}.$$

Applied twice to a position vector on the rotating Earth, it gives the familiar Coriolis and centripetal terms:

$$\mathbf{a}_I = \mathbf{a}_E + 2\boldsymbol{\Omega}_E \times \mathbf{v}_E + \boldsymbol{\Omega}_E \times (\boldsymbol{\Omega}_E \times \mathbf{r}), \quad \boldsymbol{\Omega}_E = 7.29 \times 10^{-5} \text{ rad/s}.$$

What this simulator chooses: flat, non-rotating NED

The world frame is **North-East-Down** with its origin at the home point on a flat floor (the floor is $\mathbf{z} = \mathbf{0}$, and up is negative $\mathbf{z}$). It is treated as inertial, so both extra terms above are dropped. For ATLAS that is a very good approximation:

TERM	SIZE FOR AN INDOOR HOVER	COMPARED WITH G
Coriolis $2\boldsymbol{\Omega}_E \mathbf{v}$, at 2 m/s	$2.9 \times 10^{-4} \text{ m/s}^2$	$3 \times 10^{-5} g$
Centripetal $\boldsymbol{\Omega}_E^2 \mathbf{R}_E$	0.034 m/s^2	0.3 % of g, constant, so it is absorbed into the gravity value
Change of gravity direction over 100 m	0.0009°	negligible

Gravity is a constant vector $\mathbf{g} = (0, 0, 9.80665) \text{ m/s}^2$ in NED. Air density is constant at 1.225 kg/m^3 . The only place the spherical Earth appears is in the GPS model, which converts NED position to latitude and longitude with a small-offset (equirectangular) mapping around the home point:

$$\varphi = \varphi_0 + \frac{n}{R_E}, \quad \lambda = \lambda_0 + \frac{e}{R_E \cos \varphi_0}, \quad h = h_0 - d, \quad R_E = 6\,371\,000 \text{ m}.$$

Gravity and frames: `dynamics/rigid_body.py` . GPS and magnetic field: `sensors/models.py` (`Home` , dipole field with inclination $\arctan(2 \tan \varphi)$).

The frames that move with the aircraft

Every vector in the code is expressed in one of these frames. Most bugs found in this project so far were a vector in the wrong one.

FRAME	AXES AND ORIGIN	USED FOR
World (NED)	North, East, Down; home point on the floor	Position, velocity, gravity, wind, floor contact
Structural = simulator body frame	Forward, Right, Down (FRD), fixed to the CAD; moments are taken about the CG at <code>mass.cg</code>	Rotor positions and axes, inertia tensor, forces, angular velocity, IMU output
Hover frame = PX4 body frame	The structural frame pitched nose-up by θ_h (23.85° on the 50/65/50 airframe)	What PX4 calls "level". PX4 is told with <code>SENS_BOARD_Y_OFF = θ_h</code>
Rotor frame	Unit thrust direction $\hat{\mathbf{a}}_i$ and position $\mathbf{r}_i$ in the structural frame	Thrust and reaction torque; exported to PX4 as <code>CA_ROT0Ri_*</code> in the hover frame
Sensor frames	IMU in the structural frame; H-FLOW fixed to the frame and tilted 20.4°, so 3.45° off the hover vertical	What each sensor reports

Converting from structural to hover coordinates is a single pitch rotation:

$$\mathbf{v}_{\text{hover}} = \mathbf{R}_h \mathbf{v}_{\text{struct}}, \quad \mathbf{R}_h = \begin{bmatrix} \cos \theta_h & 0 & \sin \theta_h \\ 0 & 1 & 0 \\ -\sin \theta_h & 0 & \cos \theta_h \end{bmatrix}, \quad \mathbf{R}_{\text{PX4}}^N = \mathbf{R}_{\text{struct}}^N \mathbf{R}_h^T.$$

IN PLAIN WORDS

Hover frame and rotor frame

Hover frame. Picture ATLAS hovering still. Its structure is tilted 23.85° nose-up, because that is the angle at which the nine tilted jets balance. PX4 wants "level" to mean "hovering still", so it uses a set of axes that is tilted along with the aircraft: in a hover, the hover frame's x axis is horizontal and its z axis points straight down. All of PX4's targets (roll 0, pitch 0, stick centred) refer to this frame. If PX4 used the structural axes instead, a still hover would need a permanent 23.85° pitch command.

Rotor frame. Each fan has its own small description: the point where its push acts, $\mathbf{r}_i$, and the direction of the push, $\hat{\mathbf{a}}_i$, a unit arrow. Here is what that looks like for two V3 fans on the 50 / 65 / 50 foils:

FAN	PUSH DIRECTION, STRUCTURAL AXES	SAME PUSH, HOVER AXES	MEANING IN A HOVER
Inner foil, exit 50°	(0.643, 0, -0.766)	(0.278, 0, -0.960)	96 % up, 28 % forward
Middle foil, exit 65°	(0.423, 0, -0.906)	(0.021, 0, -1.000)	almost straight up

PX4's motor mixer is given the hover-axes numbers (`CA_ROT0Ri_AX/AY/AZ`) and the fan positions from the CG in hover axes (`CA_ROT0Ri_PX/PY/PZ`). From those it works out which fans to speed up for a given roll, pitch or yaw command.

IN PLAIN WORDS

Pitch rotation between frames

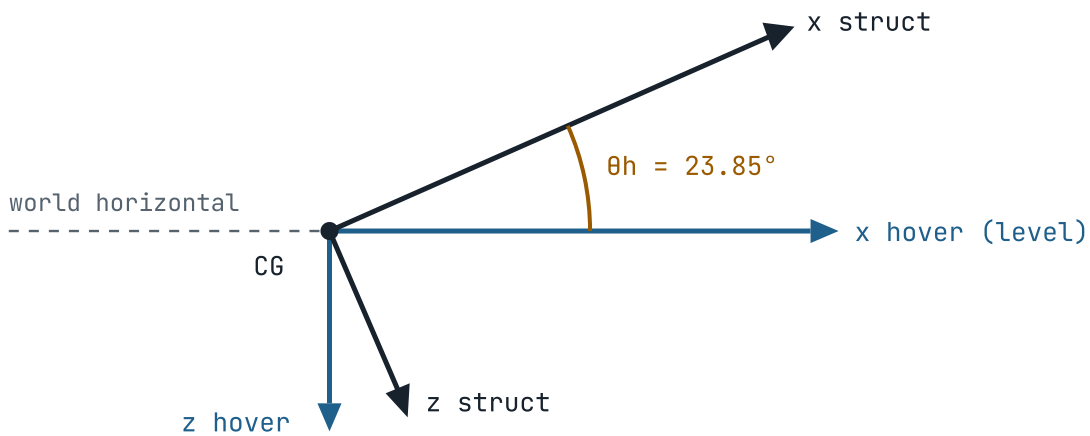
The same arrow has different numbers in different frames, the way one direction can be "straight ahead" to the pilot and "north-east" on the map. Going from structural numbers to hover numbers means turning the axes by 23.85° about the wing axis (y). The y component does not change; x and z mix through the cosine and sine of the angle:

$$\begin{aligned} v_{x,\text{hover}} &= \cos \theta_h v_x + \sin \theta_h v_z = 0.915 v_x + 0.404 v_z \\ v_{z,\text{hover}} &= -\sin \theta_h v_x + \cos \theta_h v_z = -0.404 v_x + 0.915 v_z \end{aligned}$$

Example 1. The structural forward axis $(1, 0, 0)$ becomes $(0.915, 0, -0.404)$ in hover axes: in a hover, the nose points 0.40 upward.

Example 2, measured on the board on 30 Sep. Parked at -16° , the accelerometer's raw reading, in structural axes, was $(-2.71, 0.04, -9.43) \text{ m/s}^2$. After PX4 applied the 23.85° board rotation it read $(-6.26, 0.13, -7.52)$. The formula above gives $(-6.29, 0.04, -7.53)$: the same rotation, applied by hand.

Going back is the opposite turn, R_h^T (the transpose of a rotation matrix is its inverse). And to get from the hover frame all the way to the room, chain the two turns: $R_{\text{PX4}}^N = R_{\text{struct}}^N R_h^T$.



In a steady hover the structure sits nose-up by θ_h , so its x axis points up the slope and the hover frame is level. PX4 controls the hover frame; the simulator integrates the structural one. The two are always one fixed pitch rotation apart.

IN PLAIN WORDS

The flow gyro frame mismatch found on 30 Sep

1. **What the H-FLOW sees.** A downward camera sees the floor pattern move for two reasons: the aircraft slides (what we want to measure), or the aircraft rotates (turning the camera makes the floor appear to move even if the aircraft stays in place).
2. **How PX4 separates them.** It subtracts the rotation part using a gyro reading from the same moment. For that to work, the gyro and the camera must use the same axes.
3. **Where it went wrong.** PX4's main sensor path rotates the raw gyro by `SENS_BOARD_Y_OFF` into hover axes. Its optical-flow module, when the flow sensor sends no gyro of its own, takes the raw gyro instead, still in structural axes.
4. **What that does.** A pure yaw turn at rate r about the hover vertical reads, in structural axes, as $(-r \sin 23.85^\circ, 0, r \cos 23.85^\circ) = (-0.40 r, 0, 0.91 r)$. So 40 % of every yaw rate looks like a roll rate to the flow compensation, and every turn is misread as a small sideways slide.
5. **Fix in the simulator.** The simulated H-FLOW now sends its own gyro reading, in its own axes and over the same time window as the image. PX4 uses it directly.
6. **On the real aircraft.** If the real H-FLOW sends no gyro, set `EKF2_OF_GYR_SRC = 1`. The estimator then uses its own gyro, which is already in hover axes.

Correction to the first version of this report: this mismatch was real but it was not the main cause of the heading jitter seen in HITL. The main cause was the simulated flow noise, which was scaled wrong and came out 7 times too large (0.14 rad/s per reading instead of 0.02). Fixing the noise brought the heading jitter in SITL from 1.7° to 0.13°; the gyro fix alone changed nothing measurable.

2

Quaternions

What problem they solve

An attitude is a rotation that takes vectors from the body frame to the world frame. It has three degrees of freedom, and the obvious three-number description is Euler angles (yaw ψ , pitch θ , roll ϕ , applied in that order). Their weakness is a singularity: at $\theta = \pm 90^\circ$ yaw and roll rotate about the same axis and one degree of freedom is lost (gimbal lock). Near it, the kinematic equations for the Euler-angle rates divide by $\cos \theta$ and blow up:

$$\dot{\phi} = p + (q \sin \phi + r \cos \phi) \tan \theta, \quad \dot{\psi} = \frac{q \sin \phi + r \cos \phi}{\cos \theta}.$$

ATLAS normally lives between -16° (parked) and $+24^\circ$ (hover), but it tumbles in crash tests, it rotates its nose through 40° on the ground, and the hover frame is itself pitched, so a representation without singularities is the safe choice. The unit quaternion is the standard one: four numbers with one constraint, no singularity, and cheap to propagate.

IN PLAIN WORDS

The 90° singularity (gimbal lock)

Euler angles describe an attitude as three turns done one after the other: first **yaw** about the vertical, then **pitch** about the new wing axis, then **roll** about the new nose axis.

Now pitch the nose up to exactly 90°. The nose points straight up, so the roll axis (the nose) has become the same line as the yaw axis (the vertical). Rolling and yawing now do the same thing. Infinitely many pairs of yaw and roll describe the same attitude, and the three numbers can no longer tell them apart. One degree of freedom has disappeared.

In the equations this shows up as a division by $\cos \theta$:

$$\dot{\psi} = \frac{q \sin \phi + r \cos \phi}{\cos \theta}.$$

- At $\theta = 60^\circ$: $1/\cos \theta = 2$. A body yaw rate of 1°/s needs the yaw angle to change at 2°/s.
- At $\theta = 89^\circ$: $1/\cos \theta = 57$. The same 1°/s needs 57°/s. Small numerical errors are multiplied by 57.
- At $\theta = 90^\circ$: division by zero. The simulation would fail.

A quaternion has no such point. Every attitude has exactly two quaternions ($\mathbf{q}$ and $-\mathbf{q}$, which mean the same thing), and nothing in its equations divides by zero. ATLAS hovers at 24°, far from 90°, but crash tests tumble through it, and one code path (the JSBSim readback) still goes through Euler angles.

Definition and convention used here

A rotation by angle ϑ about the unit axis $\hat{\mathbf{n}}$ is

$$\mathbf{q} = \begin{bmatrix} w \\ x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \cos(\vartheta/2) \\ \hat{\mathbf{n}} \sin(\vartheta/2) \end{bmatrix}, \quad \|\mathbf{q}\| = 1.$$

The code stores it **scalar first** $[w, x, y, z]$, uses the **Hamilton** product, and the quaternion rotates **body (structural FRD) vectors into NED**. The equivalent rotation matrix, so that $\mathbf{v}^N = \mathbf{R}(\mathbf{q}) \mathbf{v}^B$, is

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}.$$

IN PLAIN WORDS

Body-to-NED unit quaternion

A quaternion is four numbers that store *which way the aircraft is pointing*. Any orientation can be reached from "level, facing north" by one single turn about some axis. The quaternion stores that turn: the first number is the cosine of half the angle, the other three are the axis scaled by the sine of half the angle.

- **Unit** means the four numbers squared add up to 1: $w^2 + x^2 + y^2 + z^2 = 1$. That is what makes it a pure rotation. Rounding errors slowly break this, which is why the code rescales it every step.
- **Body-to-NED** means it is a recipe that takes an arrow described in the aircraft's own axes (forward, right, down) and tells you where that arrow points in the room (north, east, down).

ATTITUDE	QUATERNION $[w, x, y, z]$	WHERE THE NOSE POINTS (NED)
Level, facing north	[1, 0, 0, 0]	(1, 0, 0): north
Hovering, 23.85° nose-up, facing north	[0.978, 0, 0.207, 0]	(0.915, 0, -0.404): north and up
Parked, 16° nose-down	[0.990, 0, -0.139, 0]	(0.961, 0, 0.276): north and down
Level, facing east	[0.707, 0, 0, 0.707]	(0, 1, 0): east

Read the hover line: a turn of 23.85° about the right wing (the y axis) gives $w = \cos(11.9^\circ) = 0.978$ and $y = \sin(11.9^\circ) = 0.207$. The nose then points 0.404 "up" (negative down) for every 0.915 north. Half angles are used because that makes the multiplication rule below work; a side effect is that $\mathbf{q}$ and $-\mathbf{q}$ are the same attitude.

IN PLAIN WORDS

Hamilton product

The Hamilton product $\otimes$ is the rule for multiplying two quaternions. Writing each one as a number plus an arrow, $\mathbf{a} = (a_0, \vec{a})$ and $\mathbf{b} = (b_0, \vec{b})$:

$$\mathbf{a} \otimes \mathbf{b} = (a_0 b_0 - \vec{a} \cdot \vec{b}, a_0 \vec{b} + b_0 \vec{a} + \vec{a} \times \vec{b}).$$

It is what lets quaternions do rotation work:

- **Chaining turns.** Turning by $\mathbf{q}_1$ and then by $\mathbf{q}_2$ (measured in the already-turned axes) is one turn $\mathbf{q}_1 \otimes \mathbf{q}_2$. Example: the hover-frame attitude is the structural attitude chained with a fixed -23.85° turn about y.
- **Rotating an arrow.** $(0, \vec{v}^N) = \mathbf{q} \otimes (0, \vec{v}^B) \otimes \mathbf{q}^*$, where $\mathbf{q}^* = (w, -x, -y, -z)$ is the turn undone. This gives the same answer as the matrix $R(\mathbf{q})$.
- **Order matters.** Because of the cross product, $\mathbf{a} \otimes \mathbf{b} \neq \mathbf{b} \otimes \mathbf{a}$. Same as with a book: yaw it 90° then pitch it 90°, and you end up somewhere different than pitching first.

Why the name matters. There are two conventions in use. Hamilton's (used here and in PX4) has $\mathbf{i} \mathbf{j} = \mathbf{k}$; the "JPL" convention used in some spacecraft software has $\mathbf{i} \mathbf{j} = -\mathbf{k}$, which flips the direction of every chained rotation. Both this simulator and PX4 use Hamilton with the scalar first, so quaternions pass between them unchanged.

How the attitude evolves

The body angular velocity $\omega^B = (p, q, r)$ comes out of Euler's equation (section 3). The attitude follows from the quaternion kinematic equation, the quaternion form of $\dot{R} = R[\omega^B]_\times$:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{bmatrix} 0 \\ \omega^B \end{bmatrix} = \frac{1}{2} \begin{bmatrix} -xp - yq - zr \\ wp + yr - zq \\ wq - xr + zp \\ wr + xq - yp \end{bmatrix}.$$

Each 1 ms substep takes one explicit step with the angular velocity already updated in that substep, then divides by the norm, because a finite step slowly pulls $\mathbf{q}$ off the unit sphere:

$$\mathbf{q}_{k+1} = \frac{\mathbf{q}_k + \dot{\mathbf{q}}(\mathbf{q}_k, \omega_{k+1}) \Delta t}{\|\mathbf{q}_k + \dot{\mathbf{q}}(\mathbf{q}_k, \omega_{k+1}) \Delta t\|}, \quad \Delta t = 1 \text{ ms}.$$

With rates of a few rad/s, the rotation per step is a few milliradians, so the first-order step plus renormalisation is accurate to well under a microradian per step.

Where Euler angles still appear

Euler angles are only produced for people and for interfaces that expect them, using the ZYX sequence:

$$\phi = \text{atan2}(2(wx + yz), 1 - 2(x^2 + y^2)), \quad \theta = \arcsin(2(wy - zx)), \quad \psi = \text{atan2}(2(wz + xy), 1 - 2(y^2 + z^2)).$$

- The UI and the nose-lift controller read **structural** pitch (so the parked aircraft reads -16°).
- Tuning metrics use **hover-frame** angles, from $R_p = R(\mathbf{q}) R_h^T$, so a perfect hover reads 0° .
- The JSBSim backend reads Euler angles back from JSBSim and rebuilds the quaternion, which reintroduces the $\pm 90^\circ$ singularity on that path only.
- In SITL the quaternion is also sent to PX4 as `HIL_STATE_QUATERNION` (ground truth for logging; PX4's estimator does not use it). It carries the structural attitude, not the hover-frame one, which is worth remembering when comparing logs.

Quaternion algebra: `dynamics/quaternion.py`. Use: `dynamics/rigid_body.py` (`rotmat`, `euler`, hover-frame Euler).

3

State vector

What counts as state

A state is the smallest set of numbers that, together with the inputs, determines the future. For the simulated ATLAS V3 it is:

$$\mathbf{x} = \left[\underbrace{\mathbf{p}^N}_3 \underbrace{\mathbf{v}^N}_3 \underbrace{\mathbf{q}}_4 \underbrace{\omega^B}_3 \underbrace{\bar{\Omega}_1 \dots \bar{\Omega}_9}_9 \right]^T \quad (22 \text{ numbers, } 21 \text{ degrees of freedom})$$

BLOCK	MEANING	FRAME
$\mathbf{p}^N$	Position of the CG	NED
$\mathbf{v}^N$	Velocity of the CG	NED
$\mathbf{q}$	Attitude, body to NED (4 numbers, 3 degrees of freedom)	structural $\rightarrow$ NED
$\boldsymbol{\omega}^B$	Angular velocity	structural body
$\bar{\Omega}_i$	Normalised fan speed of fan i , 0 to 1	scalar

Some things look like state but are not stored as state:

- **Leg contact** is algebraic: the penetration of each foot is computed from $\mathbf{p}$ and $\mathbf{q}$ every substep, so legs add no states.
- **Outputs** such as the specific force, total thrust and "on ground" flag are recomputed each step.
- **Controllers outside the body** keep their own memory: the nose-lift controller (an integrator and a state machine) and the vibration model (fan phase angles).
- There is **no battery state**; thrust capability does not fall as the pack drains.

Equations of motion

The model is a single rigid body with fans that only push. Translation is written in the inertial NED frame and rotation in the body frame, which is where the inertia tensor is constant:

$$\begin{aligned}
 \dot{\mathbf{p}}^N &= \mathbf{v}^N \\
 m \dot{\mathbf{v}}^N &= \mathbf{R}(\mathbf{q}) \mathbf{F}^B + m g \hat{\mathbf{e}}_3 + \mathbf{F}_{\text{contact}}^N \\
 \dot{\mathbf{q}} &= \frac{1}{2} \mathbf{q} \otimes [0, \boldsymbol{\omega}^B] \\
 I \dot{\boldsymbol{\omega}}^B &= \mathbf{M}^B - \boldsymbol{\omega}^B \times I \boldsymbol{\omega}^B \\
 \dot{\bar{\Omega}}_i &= \frac{u_i - \bar{\Omega}_i}{\tau_i}, \quad \tau_i = 0.12 \text{ s (spin-up and spin-down set separately)}
 \end{aligned}$$

The fourth line is Euler's equation for a rigid body in principal or non-principal body axes. The gyroscopic term $\boldsymbol{\omega} \times I \boldsymbol{\omega}$ is kept; with products of inertia it couples the axes.

IN PLAIN WORDS

Reading the equations of motion, line by line

Notation: a dot on top means "rate of change over time". Superscript N means the arrow's numbers are in NED (room) axes, B means in body axes. $\hat{\mathbf{e}}_3 = (0, 0, 1)$ is the unit arrow pointing down.

1. $\dot{\mathbf{p}}^N = \mathbf{v}^N$: the position changes at the speed of the velocity. Nothing more.
2. $m \dot{\mathbf{v}}^N = \mathbf{R}(\mathbf{q}) \mathbf{F}^B + m g \hat{\mathbf{e}}_3 + \mathbf{F}_{\text{contact}}^N$: Newton's law. Mass times acceleration equals the sum of three pushes: the fans and drag (worked out in body axes, then turned into room axes by $\mathbf{R}(\mathbf{q})$), the weight ($m g$ straight down, about 116 N), and the floor pushing on the feet.
3. $\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes [0, \boldsymbol{\omega}^B]$: the attitude changes according to how fast the aircraft spins, measured in its own axes.
4. $I \dot{\boldsymbol{\omega}}^B = \mathbf{M}^B - \boldsymbol{\omega}^B \times I \boldsymbol{\omega}^B$: the rotational version of Newton's law. Inertia times angular acceleration equals the twisting moments. The extra term $\boldsymbol{\omega} \times I \boldsymbol{\omega}$ is the gyroscopic effect: a body spinning about an axis that is not one of its natural (principal) axes pushes itself towards other axes.
5. $\dot{\bar{\Omega}}_i = (u_i - \bar{\Omega}_i)/\tau_i$: each fan's speed moves towards its command, explained in the next box.

IN PLAIN WORDS

Fan spool and spool lag, first order with $\tau = 0.12$ s

"Spool" is borrowed from jet engines and just means the fan's rotational speed. "Spooling up" is speeding up. $\bar{\Omega}_i$ is fan i 's speed as a fraction of full speed, from 0 to 1, and u_i is what PX4 commands.

Lag. A fan cannot change speed instantly: its rotor has inertia and the motor's current is limited. The simplest model of that is a **first-order lag**: the speed changes at a rate proportional to how far it still is from the command. Close to the target it slows down, far from it it moves fast. For a step in command the solution is an exponential:

$$\bar{\Omega}(t) = u + (\bar{\Omega}_0 - u) e^{-t/\tau}.$$

τ (tau), the **time constant**, says how fast. After one τ the fan has covered 63 % of the step; after three, 95 %:

TIME AFTER THE STEP	0.12 S	0.24 S	0.36 S	0.60 S
Share of the speed change done	63 %	86 %	95 %	99 %

In the simulator each 1 ms step closes **1/120**, about 0.8 %, of the remaining gap. Spin-down has its own time constant, since a fan coasts down differently from how it is driven up.

Why it matters. The lag is a delay inside every control loop. For a wobble at 2 Hz the fans respond **$\arctan(2\pi \cdot 2 \cdot 0.12) = 56^\circ$** late, which limits how stiff PX4's gains can be before the loop starts to oscillate. The 0.12 s value is a default for ducted fans and has not been measured on the ATLAS fans; a bench step test would replace it.

IN PLAIN WORDS

What "translation is written in NED" means

"Writing an equation in a frame" means choosing which three axes you use to list the components of the arrows in it. The physics is the same; only the bookkeeping changes.

- **Translation** is how the CG moves through the room: position and velocity.
- **Rotation** is how the body spins about its CG: angular velocity and attitude.

Translation in NED axes, because there Newton's law has its plain form **$m\mathbf{a} = \mathbf{F}$** (NED is treated as inertial), gravity is the same **$(0, 0, g)$** everywhere, the floor is simply **$z = 0$** , and GPS works in the same axes. In body axes the same equation picks up an extra **$\boldsymbol{\omega} \times \mathbf{v}$** term (from the transport theorem), and the gravity components would keep changing as the aircraft tilts.

Rotation in body axes, because the inertia tensor describes how the mass is spread around the axes. Every part is bolted to the airframe, so in body axes those are fixed numbers. In NED axes the tensor changes every time the aircraft turns, **$\mathbf{I}^N = \mathbf{R} \mathbf{I}^B \mathbf{R}^T$** . The price for using body axes is the extra **$\boldsymbol{\omega} \times \mathbf{I} \boldsymbol{\omega}$** term, which is exactly what Euler's equation adds.

The two halves are joined by **$\mathbf{R}(\mathbf{q})$** : forces are worked out in body axes, where the fans are, and turned into NED for the translation equation.

The inertia tensor includes the products of inertia, built from the labelled CAD parts with the parallel-axis theorem:

$$I = \begin{bmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{xy} & I_{yy} & -I_{yz} \\ -I_{xz} & -I_{yz} & I_{zz} \end{bmatrix}, \quad I = \sum_k \left(I_k + m_k (\|\mathbf{r}_k\|^2 E - \mathbf{r}_k \mathbf{r}_k^T) \right),$$

where $\mathbf{r}_k$ is part k 's position from the CG. For the V3 model with the 50 / 65 / 50 foils this gives about $I \approx \text{diag}(0.57, 0.93, 1.31) \text{ kg}\cdot\text{m}^2$ with $I_{xz} \approx 0.08$, for 11.8 kg.

IN PLAIN WORDS

Parallel-axis theorem

Moment of inertia measures how hard something is to spin about an axis. It depends on how much mass there is and, much more, on how far that mass is from the axis: distance counts squared.

Each CAD part comes with a small inertia about its *own* centre. To use it for the whole aircraft, it has to be moved to the aircraft's CG. The parallel-axis theorem says how: for an axis parallel to one through the part's own centre, at distance d ,

$$I = I_{\text{own centre}} + m d^2.$$

The tensor form shown above does the same for all three axes at once, and also produces the cross terms (products of inertia).

Worked example. One of V3's six batteries, 0.77 kg, about 15×5 cm, sitting 0.25 m ahead of the CG:

- its own pitch inertia, as a small box: $\frac{1}{12} 0.77 (0.15^2 + 0.05^2) \approx 0.0016 \text{ kg}\cdot\text{m}^2$;
- the parallel-axis part: $0.77 \times 0.25^2 \approx 0.048 \text{ kg}\cdot\text{m}^2$, 30 times more;
- the whole aircraft's pitch inertia is about $0.93 \text{ kg}\cdot\text{m}^2$, so that one battery alone adds about 5 %.

That is why moving batteries changes how quickly ATLAS can pitch and yaw. The products of inertia come from the same sum: $I_{xz} = \sum m_k x_k z_k$ is not zero when heavy parts sit forward-and-low or aft-and-high, as on V3 ($I_{xz} \approx 0.08$), and it couples pitch and yaw.

The force and moment sums

Everything is added up in the structural frame about the CG. With $\mathbf{r}_i$ the point where fan i 's jet force acts:

$$\begin{aligned} T_i &= T_{\text{eff},i} s_i \bar{\Omega}_i^2 \\ \mathbf{F}^B &= \sum_i T_i \hat{\mathbf{a}}_i - \sum_i \dot{m}_i \mathbf{V}_i + \mathbf{F}_{\text{body drag}} + \mathbf{F}_{\text{wings}} \\ \mathbf{M}^B &= \sum_i T_i (\mathbf{r}_i \times \hat{\mathbf{a}}_i - k_{m,i} \hat{\mathbf{a}}_i) - \sum_i \mathbf{r}_i \times \dot{m}_i \mathbf{V}_i + \mathbf{M}_{\text{body drag}} + \mathbf{M}_{\text{wings}} \end{aligned}$$

s_i is a health factor for motor-out tests, $k_m = 0.002$ m the reaction-torque coefficient, and the $\dot{m}_i \mathbf{V}_i$ terms are the ram drag of section 6. V3 has no wings, so that term is zero.

The legs are penalty springs. For a foot with penetration δ into the floor:

$$f_n = \max(k \delta + c \max(\dot{\delta}, 0), 0), \quad \mathbf{f}_t = -\mu f_n \frac{\mathbf{v}_h}{\max(\|\mathbf{v}_h\|, 0.2 \text{ m/s})}.$$

Damping only acts in compression, and it is limited so that a stiff leg on a light vehicle cannot destabilise the integrator. Friction is Coulomb above 0.2 m/s and viscous below it.

IN PLAIN WORDS

Leg contact is algebraic

Some quantities need their history to be known. The velocity at the next step depends on the velocity now plus the acceleration; that is why it is a **state** and has its own differential equation.

Other quantities can be worked out fresh at any moment from the states alone. Those are **algebraic**: just a formula, no memory. Leg forces are like that. Every step the code:

1. works out where each foot is, from the position $\mathbf{p}$ and attitude $\mathbf{q}$;
2. if a foot is below the floor, measures how far: the penetration δ ;
3. sets the force from that and how fast it is changing: $\mathbf{f}_n = k\delta + c\dot{\delta}$.

Example: 11.8 kg resting on 3 feet. Each foot carries about 38.7 N; with this airframe's $k = 1500 \text{ N/m}$ per leg that is about 2.6 cm of static sink.

What follows from it being algebraic:

- the legs act as massless springs that respond instantly;
- no compression is stored between steps; it is recomputed from where the body is;
- friction has no "stuck" memory either, so a parked aircraft can creep very slowly instead of sticking.

A multibody engine such as Chrono, with each leg as its own body, would turn the legs into states.

How it is integrated

The integrator is semi-implicit (symplectic) Euler at $\Delta t = 1 \text{ ms}$:

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{a}_k \Delta t, \quad \mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_{k+1} \Delta t, \quad \boldsymbol{\omega}_{k+1} = \boldsymbol{\omega}_k + \dot{\boldsymbol{\omega}}_k \Delta t, \quad \mathbf{q}_{k+1} = \mathcal{N}(\mathbf{q}_k + \dot{\mathbf{q}}(\mathbf{q}_k, \boldsymbol{\omega}_{k+1}) \Delta t).$$

Using the new velocity for the position update makes the scheme conserve energy well for oscillators such as the legs, where plain explicit Euler would slowly pump energy in. It stays stable while $\omega_n \Delta t < 2$. The stiffest oscillator here is a leg: $k = 1500 \text{ N/m}$ on about 3.9 kg, so $\omega_n \approx 20 \text{ rad/s}$ and $\omega_n \Delta t \approx 0.02$, far inside the limit. If the state ever goes non-finite, the vehicle is put back on its legs and the event is logged.

IN PLAIN WORDS

Reading the integrator

The computer cannot solve the equations continuously, so it takes small steps of $\Delta t = 1 \text{ ms}$. k is the step number: $\mathbf{v}_k$ is the velocity now, $\mathbf{v}_{k+1}$ one millisecond later.

1. New velocity = old velocity + acceleration $\times 1 \text{ ms}$.
2. New position = old position + *new* velocity $\times 1 \text{ ms}$. Using the freshly updated velocity, instead of the old one, is what "semi-implicit" means.
3. New spin rate = old spin rate + angular acceleration $\times 1 \text{ ms}$.
4. New attitude = old attitude nudged by the *new* spin rate, then $\mathcal{N}(\cdot)$ rescales it back to length 1.

Why the order matters. Take one leg as a spring with $\omega_n \Delta t \approx 0.02$. With plain explicit Euler (old velocity in step 2) each step multiplies the bounce amplitude by $\sqrt{1 + 0.0195^2} = 1.00019$. That looks harmless, but over 1000 steps (one second) it grows by $e^{0.19} = 1.21$, about 20 % more bounce from nothing. The semi-implicit order keeps the amplitude bounded, as long as $\omega_n \Delta t < 2$.

From state to sensors

An accelerometer does not measure acceleration. It measures **specific force**, acceleration minus gravity, in its own frame. The simulator derives it from the motion actually integrated, so it is exactly consistent with the trajectory:

$$\mathbf{f}^B = R(\mathbf{q})^T \left(\frac{\mathbf{v}_{k+1} - \mathbf{v}_k}{\Delta t} - g \hat{\mathbf{e}}_3 \right), \quad \boldsymbol{\omega}_{\text{gyro}} = \boldsymbol{\omega}^B + \mathbf{b}_g + \mathbf{n}_g + \boldsymbol{\omega}_{\text{vib}}.$$

At rest this reads $(0, 0, -g)$: the floor pushes up, so the sensor reports "up" as negative z. Noise, bias and a fan vibration model are added on top. The vibration model reaches the sensors only; it does not shake the simulated airframe.

IN PLAIN WORDS

Reading the sensor equations

Accelerometer, $\mathbf{f}^B$. Take the actual change of velocity over the step (that is the true acceleration), subtract gravity, and turn the result into body axes with R^T . An accelerometer cannot feel gravity directly; it feels everything that stops it from falling.

- Sitting on the floor: acceleration 0, minus gravity $(0, 0, g)$, gives $(0, 0, -g)$. It reads "pushed up by 1 g".
- In free fall: acceleration equals gravity, so it reads 0.
- Hovering at 23.85° in structural axes: $R^T(0, 0, -g)$ gives $(3.97, 0, -8.97) \text{ m/s}^2$. The x axis "feels" part of the hover thrust because the structure is tilted.

Gyroscope, $\boldsymbol{\omega}_{\text{gyro}}$. The true spin rate plus a bias $\mathbf{b}_g$ (a small constant offset, as real gyros have), noise $\mathbf{n}_g$ (random jitter, $\sigma = 0.002 \text{ rad/s}$), and $\boldsymbol{\omega}_{\text{vib}}$ (the fan buzz from the vibration model).

Two state vectors, not one. The simulator holds the true state $\mathbf{x}$. PX4 runs its own estimator (EKF2) with a different state of about 24 numbers: attitude, velocity, position, gyro and accelerometer biases, Earth and body magnetic fields, and wind. It only ever sees the sensor outputs. Almost every HITL problem on 30 Sep was a gap between the two: the true attitude was fine, but EKF2's estimate was off by 0.3° to 1° because of a stored accelerometer offset or noisy flow data. Comparing *estimate* with *truth* is the single most useful diagnostic the simulator offers.

4

JSBSim with Python integration

What JSBSim is

JSBSim is an open-source six-degree-of-freedom flight dynamics engine written in C++. It is the physics core of FlightGear and is used in industry and research. An aircraft is described in an XML file (mass, inertia, contact points, propulsion, aerodynamic coefficient tables) and JSBSim integrates its rigid-body motion over a rotating, oblate (WGS84) Earth, with a standard atmosphere and ground reactions. A Python package (`pip install jsbsim`) exposes the engine: you create an `FGFDMExec` , load a model, then read and write named properties such as `velocities/p-rad_sec` and advance time with `run()` .

How it is used here

JSBSim is an **optional second physics backend**, selected with `physics="jsbsim"` (CLI `--physics jsbsim` , or the physics switch in the app). Its purpose is to cross-check the Python integrator, frames and contact model with an independent engine. It does not add aerodynamic fidelity, because both backends use the same rotor and drag code.

TASK	DONE BY
6-DOF integration, gravity, Earth model	JSBSim
Atmosphere	JSBSim (the Python backend uses constant density)
Leg contact (feet as <code>STRUCTURE</code> contacts with spring, damping, friction)	JSBSim
Moment arms of the fan forces	JSBSim, from each force's location
Fan spool, thrust, reaction torque, ram drag, body drag	Python, injected every step as external forces and moments
Wings, if any	JSBSim, from coefficient tables Python generates

The Python to JSBSim bridge

- 1. Generate the model.** `generate_model` writes an aircraft XML under `~/airframe_designer/jsbsim/`: mass and inertia including products (converted to slugs and slug-ft²), each foot as a contact, one `external_reactions` force per fan in the BODY frame at the fan's position, and aerodynamic tables of C_X , C_Z , C_m against α from -180° to 180° (sampled from the strip-theory wing model at 20 m/s), plus linear side-force and damping derivatives.
- 2. Convert frames and units.** JSBSim's structural frame has x aft, y right, z up, in inches, so every point maps as $\mathbf{r}_{JSB} = (-x, y, -z) \times 39.37$.
- 3. Each step.** Python computes the fan forces from the current state, writes them into custom properties, passes the wind, and calls `run()`.
- 4. Read back.** Position (north, east, height above ground from the start point), NED velocity, body rates, and Euler angles, from which the quaternion is rebuilt. The specific force is derived the same way as in the Python backend.

IN PLAIN WORDS

FGFDMExec

`FGFDMExec` stands for "Flight Dynamics Model Executive". It is JSBSim's main object: one instance is one complete simulated aircraft. It:

- reads the aircraft XML file;
- owns all the sub-models: mass and inertia, propulsion, aerodynamics, ground reactions, external forces, atmosphere, the Earth model, and the integrator that moves the state forward;
- runs them in the right order every time step;
- keeps a **property tree**: every quantity has a path-like name, such as `velocities/p-rad_sec` or `ic/h-agl-ft`. The suffix gives the unit (feet, slugs, pounds, degrees, radians per second).

This is how the backend drives it from Python (`dynamics/jsbsim_backend.py`):

1. `fdm = jsbsim.FGFDMExec(root)`: create the executive, pointing at the folder with the generated aircraft.
2. `fdm.load_model(name)` and `fdm.set_dt(dt)`: load the XML and set the step.
3. `fdm["ic/theta-deg"] = pitch` (and latitude, height, heading), then `fdm.run_ic()`: set and apply the initial conditions.
4. Every step: write each fan's thrust into `external_reactions/rotori/magnitude`, call `fdm.run()`, and read back properties such as `velocities/v-north-fps`.

Status and limits

- The project notes call it the cross-check engine used in practice, and comparison runs are documented for the earlier ATLAS models.
- `jsbsim` is not a declared dependency and does not import in this Mac's environment, so its tests (rest, hover climb, soft legs, force and moment equivalence) are skipped here. Install it to use the backend.
- Aerodynamic tables are frozen at one reference speed, and damping derivatives are taken only at $\alpha = 0$.
- Attitude passes through Euler angles on the way back, singular at $\pm 90^\circ$ pitch.
- The model is regenerated whenever the airframe changes, and reset to an arbitrary position is not supported.
- A Gazebo backend also exists but is unfinished and not selectable.

Bridge: `dynamics/jsbsim_backend.py` . Selection: `sim/simulator.py` (`_make_body`). Comparison: `batch/compare.py` . Tests: `tests/test_jsbsim_backend.py` .

5 Project Chrono

Status in this repository

Nothing in the code, documentation, project notes or dependencies refers to Project Chrono. It is **not present** . This section explains what it is and where it would fit if it were added.

What Chrono is

Project Chrono is an open-source multibody dynamics library (C++, with Python bindings called PyChrono). Where JSBSim is built around one aircraft and its aerodynamics, Chrono is built around many connected bodies and their contacts:

- rigid bodies connected by joints (revolute, prismatic, springs and dampers), solved as a constrained system;
- two contact formulations: a penalty method with Hertz or Hunt-Crossley force laws (SMC), and a complementarity method that treats contact as a hard constraint (NSC);
- flexible bodies through finite elements (beams, shells) for structural modes;
- add-on modules for vehicles, granular terrain and fluid-structure interaction.

It has no built-in aerodynamics for aircraft. Forces from fans and air would still come from this project's Python code, applied to Chrono bodies each step, in the same way the JSBSim bridge works.

Where it would fit in this simulator

AREA	TODAY	WHAT CHRONO WOULD ADD
Ground contact	Point feet, linear spring-damper, regularised Coulomb friction	Real foot shapes, Hunt-Crossley contact $f_n = k\delta^n + c\delta^n\dot{\delta}$, proper stick-slip friction cones
Nose-lift and rest rotation	The aircraft pivots on penalty springs at its rear feet	A pivot that holds as a constraint, so the rotation from -16° to 24° and the drop back are modelled without spring bounce
Legs	Rigid lines from attachment to foot, re-solved per parked angle	Articulated or compliant legs with joints and their own masses
Structure and vibration	Analytic frame modes feeding the IMU only	Flexible booms and foil mounts whose modes come from the geometry and feed back into the flight

Is it worth adding?

Only if ground interaction becomes the limiting factor. The flights that matter most to ATLAS start and end on the ground: a nose-down park, a fan-driven rotation about the rear feet, a touchdown and a nose-lowering. Those phases are where penalty contact is weakest: the 7 to 54°/s ground bounce seen in the -16° batch was an interaction between PX4's airmode, the nose-lift hold and the spring legs. A Chrono backend would show how much of that is physical and how much comes from the leg model. In the air, Chrono would add nothing over the current rigid-body integrator.

A practical route is to treat it like JSBSim: one Chrono body with the vehicle's mass and inertia, foot contact shapes, the Python `RotorSet` applying forces through Chrono's force accumulators, and `DoStepDynamics` at 1 ms. Then run the same scenarios on both backends and compare the ground phases.

6

Coanda flow and jet enclosure aerodynamics

The physical idea on ATLAS

Each ATLAS fan sits in a duct (the enclosure) with its axis pointing forward and its jet blowing aft. The jet then flows along a curved surface, the jetfoil, which turns it downward. The aircraft is lifted by the reaction to that turned jet. Two pieces of physics are involved: how much thrust a fan in a duct produces, and how well a curved wall turns a jet.

Jet enclosure: momentum theory for a ducted fan

Treat the fan as an actuator disc of area A that adds momentum to the air passing through it. With the air drawn from rest, thrust equals the momentum flux leaving the system, $T = \dot{m} V_e$, with $\dot{m} = \rho A_e V_e$ at an exit of area A_e .

- **Open rotor:** the wake contracts to half the disc area and the far-wake speed is twice the speed through the disc, which gives the classic $P_{\text{ideal}} = T^{3/2} / \sqrt{2\rho A}$.
- **Ducted fan:** the duct sets the exit area. With an exit area ratio $\sigma = A_e / A$,

$$T = \rho \sigma A V_e^2, \quad P_{\text{ideal}} = \frac{1}{2} \dot{m} V_e^2 = \frac{T^{3/2}}{2\sqrt{\rho \sigma A}}, \quad \dot{m} = \sqrt{\rho \sigma A T}.$$

For $\sigma = 1$ (the case the code assumes), the duct needs $1/\sqrt{2} \approx 71\%$ of the open rotor's power for the same thrust, or gives $2^{1/3} \approx 1.26$ times the thrust for the same power. Part of that thrust is carried by suction on the duct lip rather than by the fan. In the simulator the ducted power formula is only a diagnostic; thrust is specified directly per fan.

Ram drag: the enclosure in a crossflow

When the vehicle moves through the air at velocity $\mathbf{V}$, the fan has to take in air that arrives with that velocity and turn it into its own jet. That costs a momentum flux $\dot{m} \mathbf{V}$, felt as a drag. The simulator applies it to every ATLAS fan, using the local air velocity at the fan including rotation:

$$\mathbf{f}_{\text{ram},i} = -\dot{m}_i \mathbf{V}_i, \quad \dot{m}_i = \sqrt{\rho A_i T_i}, \quad \mathbf{V}_i = \mathbf{v}_{\text{air}} + \boldsymbol{\omega} \times \mathbf{r}_i.$$

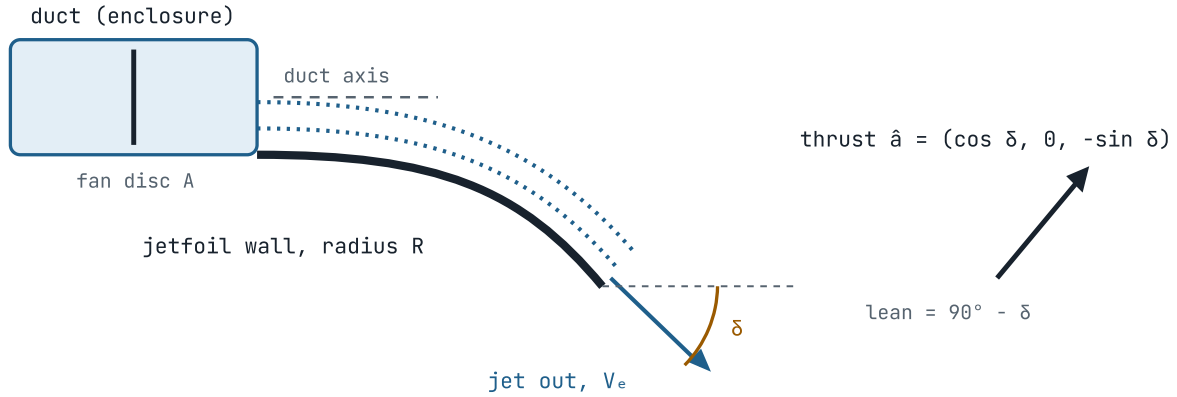
Order of magnitude for V3 in hover: about 13 N per fan through a 96 mm duct gives $\dot{m} \approx 0.34$ kg/s, so at 2 m/s each fan feels about 0.7 N, roughly 6 N for nine fans against 116 N of weight. Because it scales with $\boldsymbol{\omega} \times \mathbf{r}$ too, ram drag also damps rotation, which is part of why ATLAS yaw is sluggish.

Coanda flow: turning a jet with a curved wall

A jet blowing tangentially along a convex wall tends to follow it. To bend the streamlines around a radius R , the pressure at the wall must fall below ambient. A radial momentum balance across a jet of thickness b and speed U gives the size of that suction:

$$\frac{\partial p}{\partial n} = \frac{\rho U^2}{R} \Rightarrow \Delta p \approx \frac{\rho U^2 b}{R}.$$

The jet stays attached while that suction can be sustained against wall friction and mixing with the surrounding air. A thin, fast jet on a gently curved wall (large R/b) turns far before it separates; a thick jet on a tight curve separates early and exits at a smaller angle than the wall. If the jet stays attached, an ideal control-volume balance says the jet leaves along the trailing-edge direction with its momentum flux reduced by friction and mixing losses, and the force on the vehicle is opposite to that exit direction.



The fan blows aft along the duct axis, the jetfoil turns the jet down by the exit angle δ , and the vehicle feels a force opposite to the exit direction, tilted forward from vertical by the "lean" $90^\circ - \delta$. The jet thickness b against the wall radius R decides whether the jet really leaves at δ ; the simulator assumes it always does.

What the simulator actually does

1. **Exit angle from CAD.** For each foil, rays are cast through the Fusion model every 2.5 mm across the jet plane, and the exit angle δ is the slope of the last 5 mm of wall before the trailing edge. The v34 CAD gives $\delta = 49.4^\circ, 59.3^\circ, 69.0^\circ$ (outer, middle, inner); the tuned variant used for flights is 50 / 65 / 50.
2. **Direction.** The thrust direction is the duct axis rotated by δ about the body y axis, $\hat{a} = (\cos \delta, 0, -\sin \delta)$. That is lean $40.6^\circ / 30.7^\circ / 21^\circ$ on the CAD foils.
3. **Magnitude.** A linear turning loss, with $k_{\text{turn}} = 0.1$:

$$T_{\text{eff}} = T_{\text{max}} \max\left(0, 1 - k_{\text{turn}} \frac{\delta}{90^\circ}\right), \quad \delta = 69^\circ \Rightarrow 7.7\% \text{ loss}.$$

4. **Point of application.** One force along $\hat{a}$ acting on the jet exit line, $\mathbf{r}_i = \mathbf{r}_{\text{TE}} + h \hat{\mathbf{n}}$, half a nozzle height ($h = 15$ mm) off the trailing edge. The control-volume argument: the fan draws air from rest, so fan thrust plus the foil's reaction equals the exit momentum flux, and only their sum needs to be modelled.
5. **Ram drag** as above, for every fan.
6. **Into PX4.** PX4's control allocation is told the turned directions and the losses, in the hover frame: $\text{CA_ROT0Ri_A*} = R_h \hat{\mathbf{a}}_i$, $\text{CA_ROT0Ri_P*} = R_h (\mathbf{r}_i - \mathbf{r}_{\text{CG}})$, and CA_ROT0Ri_CT proportional to $T_{\text{eff},i}$. The hover pitch θ_h is chosen by scanning for the angle that leaves the most control headroom.

What is assumed, and what is missing

ITEM	STATUS	WHY IT MATTERS
Jet fully attached at every throttle and airspeed	assumed	If the jet separates early, the real exit angle is smaller than δ : more forward thrust, less lift, and a different hover pitch
Turning loss $k_{\text{turn}} = 0.1$	assumed	Sets each fan's share of lift; PX4 sees it through <code>CT</code>
Fan maximum thrust 36 N, $k_m = 0.002$, spool $\tau = 0.12$ s	not measured	Authority, yaw control and how fast the attitude loop can act
Nozzle height (15 mm)	not measured	Moves the force line and so the pitch moment of each fan
Separation, jet spreading, entrainment, wall curvature effects	not modelled	The core of the Coanda behaviour
Duct lip thrust, exit area ratio σ	not modelled	Only matters once thrust is computed from fan speed and geometry rather than specified
Jet-jet and jet-body interaction	not modelled	Nine jets close together can merge or deflect each other
Ground effect, suckdown, fountain	not modelled	Changes lift close to the floor, exactly where ATLAS takes off and lands, and what the H-FLOW sees through the jets

How to replace the assumptions with data

1. **Thrust stand.** One fan with its foil on a 6-axis load cell, swept in throttle. This measures T , the real force direction (so the effective exit angle), k_m and the spool time constant in one setup. It directly replaces T_{max} , k_{turn} , $\hat{a}$, k_m and τ .
2. **Flow visualisation.** Tufts or smoke on the foil show where the jet separates as throttle changes.
3. **A few CFD points.** Steady RANS on one duct and foil (for example with a $k-\omega$ SST turbulence model) at three or four throttles, and with a crossflow, to get the effective exit angle and loss as functions of throttle and airspeed.
4. **Lookup tables.** Store $\delta_{\text{eff}}(u, V)$ and $\eta(u, V)$ per foil and let the rotor model read them instead of the constant angle and linear loss. The rigid-body side does not need to change.

Fan model: `geometry/propulsion.py`, `aero/rotor_aero.py`. Foil geometry: `scripts/atlas_v3_from_fusion.py`, notes in `brain/architecture/atlas-v3-fusion.md`. PX4 export: `geometry/airframe.py` (`px4_params`).

7

How it fits together

Read from the bottom up, the six topics form one chain. Errors propagate along it, which is why the weakest link today is the first one, the jet model.

LAYER	QUESTION IT ANSWERS	IN THIS SIMULATOR	CONFIDENCE
Coanda and enclosure aerodynamics	What force does each fan produce, in which direction, at which point?	Turned jet at the CAD exit angle, linear loss, ram drag	lowest no measurements yet
State vector and equations of motion	Given those forces, how does the vehicle move?	Rigid body, Euler's equation, spring legs, fan lag, 1 kHz	high standard mechanics
Quaternions	How is orientation stored and propagated without singularities?	Unit quaternion, renormalised each step	high
World and body frames	In which coordinates is each vector, and is Newton valid there?	Flat NED as inertial; structural, hover, rotor and sensor frames	high mismatches with PX4 found and fixed
JSBSim	Does an independent engine agree?	Optional backend, same rotor code	partial checks integration and contact, not aero
Project Chrono	Are the ground phases right?	Not present	open

Suggested order of work

1. Measure one fan and foil on a thrust stand. This improves every flight result, because every force starts there.
2. Install `jsbsim` on this Mac and re-run the comparison scenarios on the V3 model, so the cross-check covers the current airframe.
3. Add ground effect as a simple height-dependent lift factor for the take-off and landing phases.
4. Consider Chrono only if the ground phases (park, rotation, touchdown) still disagree with real flights after that.

Runnable versions of every section, on the real code, are in `notebooks/` (start with `00_index.ipynb`). Compiled from the AIRFRAME_DESIGNER code on the `atlas-v5-design` branch and the project notes in `brain/`, 30 Sep 2026. Equations are the ones the code implements; file paths are relative to `airframe_designer/` unless stated. Numbers for V3 refer to the v34 model (11.8 kg, nine fans) and its 50 / 65 / 50 foil variant.